SJÄLVSTÄNDIGA ARBETEN I MATEMATIK

MATEMATISKA INSTITUTIONEN, STOCKHOLMS UNIVERSITET

Gröbner Bases and Elimination in Macaulay 2

av

Christian Eriksson

2026 - No 5

Gröbner Bases and Elimination in Macaulay 2

Christian Eriksson

Självständigt arbete i matematik 15 högskolepoäng, grundnivå

Handledare: Sofia Tirabassi

2026

Abstract

Your summary goes here

Contents

1 Introduction

Hello world
[DAC10]
[LM21]

2 The Algebreic Abstraction of Geometry

Algebra is the mathematical expression of the human need for structure and abstraction. Through algebra, humans try to capture more of the mathematical, and natural, domain under their pen. The endless hunger for knowledge can express itself in various way. When using algebra as a tool, creating abstract models that capture generic structure is favored. However, these abstract models can be difficult to understand. The ability to navigate between levels of abstraction is a vital tool for any mathematician interested in applying algebreic methods.

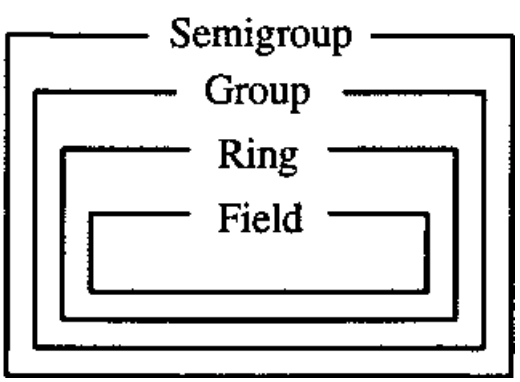
This paper assumes a degree of familiarity with algebreic structures, but the uninitiated peruser is kindly directed towards the easy-going introduction of "Introducetion to Modern Algebra" [Wei70] with bite size exercises, or alternativly, the more rigorous "Abstract Algebra" [DF03] for a deeper understanding.

As the section header suggests, this section will be primarily focused with defining the tools necessary to build an analytic bridge between geometry and algebra so that we can analyze geometric objects with algebra. A function that maps geometric objects to algebreic must be established. This function uses Affine Varietes and Ideals, along with the dictionary established in the Hilbert Strong Nullstellensatz, to accomplish that goal.

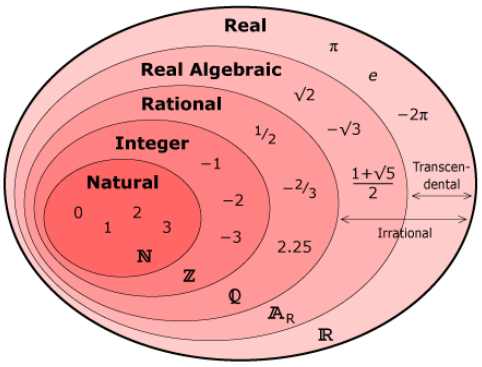
2.1 Levels of Abstraction

Abstract algebra traverses through layers of abstraction as an exercise. As an example, if we are interested in how groups act when supplemented with commutivity (Abelian groups), it would be useful to start at groups and traverse the layers of figure 1a downwards. Note that climbing upwards may not be as fruitful for this particular case.

Choosing an Abelian group that stays beteween groups and rings is $(\mathbb{R}, +)$. Here the set of real numbers is only considered with the binary operator $+$, and commutivity follows. Analyzing this group and all the properties that are associated with it is of interest. However, much use can be extracted from climbing further down the abstraction ladder and analyzing rings a. Keeping it concrete with numbers, one can consider the abelian ring $\mathbb{Z}$. Time can be spent on this layer of abstraction and knowledge can be obtained through leveraging $\mathbb{Z}$'s special properties. Going even further down the ladder, to a field, one could analyze $\mathbb{Q}$. At each layer, new intuition can be gained. Note also, that while traversing downwards in the group abstraction,



(a) Abstractions within Groups [Lat]



(b) Generalization of Numbers [Diand]

Figure 1: Traversing the ladder of Abstraction

we were climbing up and down in the generalization of numbers. Starting at the top with $\mathbb{R}$, we jumped all the way down to $\mathbb{Z}$, and then up again to $\mathbb{Q}$. It is important to consider this, as different layers of abstraction do not neccessarily have to correspond with another. Caution is required when dealing with this iterative abstract process.

2.2 Polynomials

In order to adress the problem at hand fully, we need to generalize the idea of a polynomial. This is done by compressing more and more inforamation into a single mathematical expression. A process of abstraction is necessary. We start with the monomial, continue to a polynomial, and finally consider polynomial rings. All work done is done on these polynomials rings, and this process of abstraction enables us to say something about all monomials simultaneously.

Definition 2.1. A **monomial** in $x_1, \ldots, x_n$ is a product of the form

$$x^\alpha = x_1^{\alpha_1} \cdot x_2^{\alpha_2} \cdots x_n^{\alpha_n} \tag{1}$$

where all $\alpha_1, ..., \alpha_n$ are nonnegative integers.

As the root word "mono" suggests, there is only one of these terms x^α.However, as we are creating the tools to generalize and deal with more abstract concepts, we also want to include the ability to capture any linear combination of these monomials.

Definition 2.2. A **polynomial** f in $x_1, \ldots, x_n$ with coefficients, $a_1, \ldots, a_n$, in a

field $\mathbb{K}$ is a finite linear combination of monomials.

$$f = \sum_{\alpha} a_\alpha x^\alpha, \quad a_\alpha \in \mathbb{K} \tag{2}$$

With the polynomial definitions at hand, we can construct the final abstraction for our mathematics involving polynomials, $\mathbb{K}[x] = \mathbb{K}[x_1, \ldots, x_n]$. An important note is that $\mathbb{K}[x]$ satisfies the axioms for rings and is therefore refered to as the *polynomial ring* When leveraging this powerful abstraction, we capture n dimensions of variables on any field $\mathbb{K}$. This enables more compact analysis.

2.3 Affine Spaces and Varieties

To start the bridge between the world of Algebra as we know it and geometry, there must be an established way to discuss geometry algebreically. This is where the concepts of affine spaces comes in.

Also talk about what geometry actually is in this context (points and lines and functions and so on)

This concept allows the analysor to view the geometry algebreically. However, this is not the endgoal, but an analytic step between geometry and the more powerful algebreic tools that we want to apply to geometry.

Relate also how vector spaces and affine spaces use maps.

" The geometric properties of a vector space are invariant under the group of bijective linear maps, whereas the geometric properties of an affine space are invariant under the group of bijective affine maps, and these two groups are not isomorphic." [Gal25]

Affine maps, crudely speaking, are generalizations of linear maps [Gal25]. This generalization can be viewed in different ways, but a few important examples are listed.

1. Affine spaces allow for a dynamic origin. Vector spaces do not.

2. There are more affine maps than there are linear maps.

3. Affine spaces study points in a more general matter, independently of the coordinate system chosen. Frame invariance.

2.3.1 Affine Spaces

In order to understand affine varieties, affine spaces must be understood. This coordinate-agnostic space is of interest since more abstract enteties can be modeled upon it. Let us first consider the definition:

Definition 2.3. An *affine space* is either the empty set, or a triple $< E, \vec{E}, + >$. E is a nonempty set of points while $\vec{E}$ is a vector space (of translations). The action $+$ is an operation on both E and $\vec{E}$ defined as follows: $+ : E \times \vec{E} \to E$. The following axioms are assumed for affine space:

A1 $a + 0 = a$ for every $a \in E$.

A2 $(a + u) + v = a + (u + v)$ for every $a \in E$ and every $u, v \in \vec{E}$.

A3 For any two points $a, b \in E$, there is a unique $u \in \vec{E}$ such that $a + u = b$

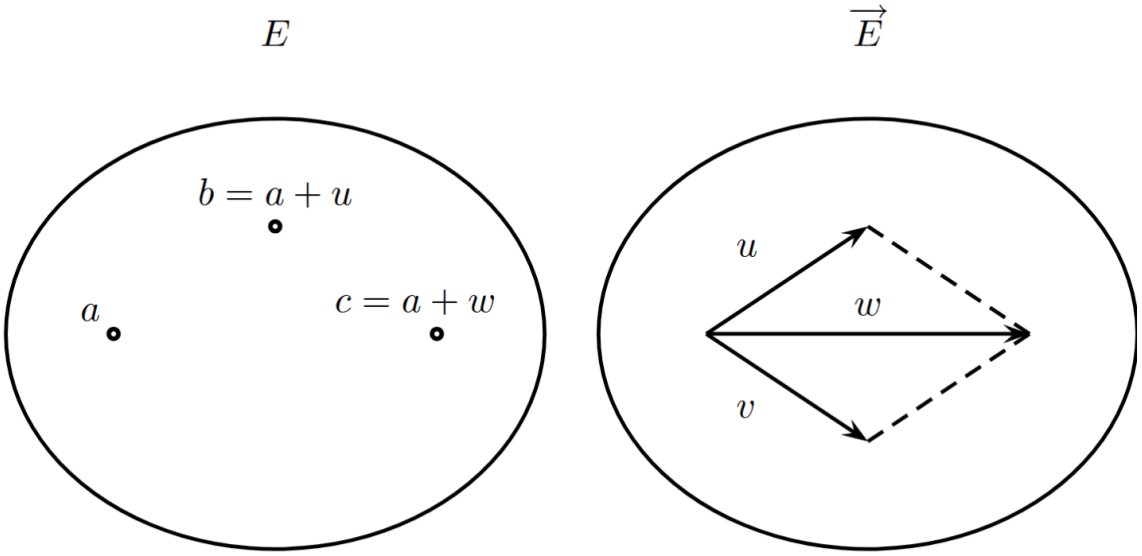


Figure 2: Intuiative understanding of affine space [Gal25]

A quick note of importance on each axiom. If you start at point a and do nothing, you should still be at a. This seems obvious and silly to include, but these are axiomatic assumptions. We must treat them with the respect they deserve. Axiom two deals with the associative property. The fact that the order of operations makes little difference to the outcome is of great interest. To simply calculations algebreically, rearanging order is always of interest. The last axiom, that of uniqueness is also of importance. Being able to assume that an unique element in $\vec{E}$ always

exists that can connect two points a and b is useful when solving these problems on a higher level of abstraction.

The definition is a bit stuffy, but the intuition should be clear when considering figure 2. Here a seperation between the vectors and the points *to which the vector is applied to* should be noted. These two can be defined independently. This seperation is exactly what enabled coordinate-agnostic property that is of interest.

A supplemental definition will be provided for the affine to clear up any confusion.

Definition 2.4. Given a field $\mathbb{K}$ and a positive integer n, define the $n-$*dimensional affine space* over $\mathbb{K}$ as:

$$\mathbb{K}^n := \left\{[\hat{x}_1, \ldots, \hat{x}_n]^T : \hat{x}_i \in \mathbb{K}, i = 1, \ldots, n\right\} \tag{3}$$

This supplement elucidates the uncertainty behind how this space is defined as all other spaces. It's vector space has translations and they are restricted to the field upon which they operate.

2.3.2 Polynomials in Affine Spaces

It is time to relate the previously discussed polynomial to affine spaces. Leveraging definition 2.4, we can relate the two. [LM21] refers to this as a "key idea" and shows that the polynomial $p \in \mathbb{K}[x]$ yields a polynomial *function* as follows:

$$p : \mathbb{K}^n \to \mathbb{K} \tag{4}$$

The function maps the multidimensional field onto the monodimension. This p takes any $\hat{x} \in \mathbb{K}$ and associates a unique $p(\hat{x})$ to it. This is done easily by substituting $x = \hat{x}$ in the polynomial p. One important note is relevant to the zeros of p when taking $\mathbb{K}$ as a infinite field (e.g. $\mathbb{Z}, \mathbb{R}$). Then p is the zero polynomial if and only if $p(\hat{x}) = 0$ for all $\hat{x} \in \mathbb{K}$.

2.3.3 Affine Varieties

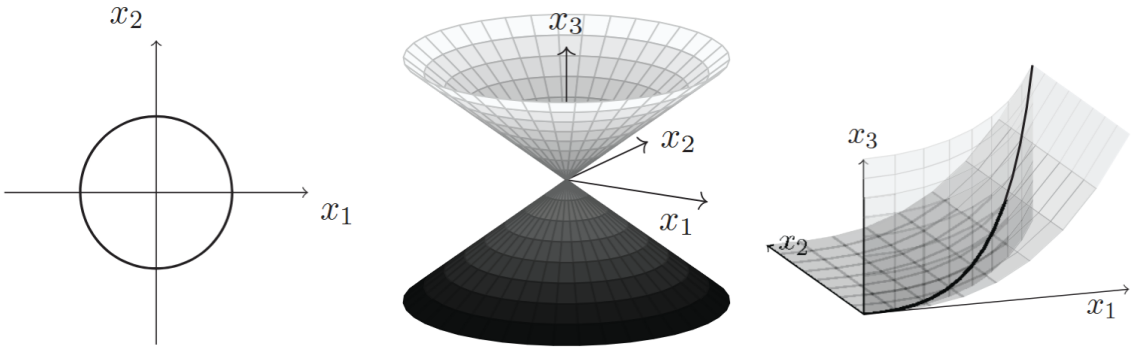
With an understanding of how the space we will operate upon will work, the first algebreic structure of geometry will be discussed: affine varieties.

Definition 2.5. Let $\mathbb{K}$ be a field, and let $f_1, \ldots, f_s$ be polynomials in $\mathbb{K}[x_1, \ldots, x_n]$.

Then, set

$$\boldsymbol{V}(f_1, \ldots, f_s) = \{(a_1, \ldots, a_n) \in \mathbb{K}^n | \ f_i(a_1, \ldots, a_n) = 0 \ \forall 1 \leq i \leq s\} \tag{5}$$

We call $\boldsymbol{V}(f_1, \ldots, f_s)$ the **affine variety** defined by $f_1, \ldots, f_s$.



(a) $\mathbf{V}(x_1^2 + x_2^2 - 1)$. (b) $\mathbf{V}(x_1^2 + x_2^2 - x_3^2)$. (c) $\mathbf{V}(x_1^2 - x_2, x_1^3 - x_3)$.

Figure 3: Exmaples of affine spaces [LM21]

In other words, the affine varity is the set of $(a_1, \ldots, a_n)$ that solve all the equations in the variety simulteniously. It is the intersection of all the polynomial's roots. Even though $\boldsymbol{V}$ might simply look like a graphed equation, figure 3a provides the base for intuition. While it may look like a regular circular graph, with your good ole' x and y coordinates, the keen observer will note that we are dealing with two dimensions, x_1, x_2 and that the graph is in fact the where this polynomial studied is equal to 0. If we extract the polynomail from the variety and set it to 0, we end up with $x_1^2 + x_2^2 = 1$. If we graph the solution to this equation, we get the resulting affine variety.

While the above example may make sense, confusion remains as to what the difference between the variety and simply graphing the equation *is*. This difference is made clear by inspecting 3c. Here, just as in 3a, we graph the solution of each individual *polynomial* seperetaly. Once we have the individual solutions, we must find their intesection, as that is where the affine variety lies. The difference between an affine variety and simply graphing the solutions to the individual equations becomes clear in higher dimensions.

2.4 Hilbert Strong Nullstellensatz

3 Gröbner Bases

3.1 Hilbert Bases

3.2 Gröbner Bases

3.3 Properties of Gröbner Bases

4 Elimination

5 Algorithms in Macaulay2

5.1 Exploring Macaulay2

After installing the Windows Subsystem for Linux, the Macaulay2 (M2) playground was ready and open. Since the main book [LM21] I was using had an introductory section to the language, the first order of business was to familiarize myself with EMACS and how the syntax of the language worked. This took quite some time as EMACS/Linux is highly unfamiliar to me and the syntax used when interfacing with the M2 console line is different from when one writes a script.

The very first script I wrote, A.5 mirrors this confusion. Even though I am creating a script. Double $*$ are being used instead of exponentials because the swedish keyboard has an odd nack for making the powers small. This lead to compiler mismatches and wasted hours. This script is a starting point in how to use the language. Important features such as which ring is being operated upon, which ordering, and how to create a Gröbner basis is touched upon. The next couple of book examples, namely A.6 and A.7 show how to construct more involved rings as well as how to evaluate functions in the ring itself. This eased the understanding of how to use rings in the lanugage.

While helpful, these book examples provided little help in how to construct a scrip that exectues any sort of meaningful code. The book displayed clips from the command line of M2, which is not the proper format for writing algorithms. So even though book exmaple A.5 and A.8 show built in ways to perform division and find the remainder, implementing a division algorithm in one and multiple dimensions seemed like an appropriate start to acclimate myself to the language.

5.2 Divisions Algorithms

Division algorithms have been of interest for mathematicians for a long time. Euclids algorithm, one of the oldest algoorithms created, has applications in for division as well. Basic arithamtic with additions, subtraction, and multiplication can be constrained to $\mathbb{Z}$, the integers, while the introduction of division neccessiates an entirely different way of interacting with numbers, $\mathbb{Q}$, the rationals. Division is useful, so a mathematical programming lanugage must be equipped with ways to handle division. Macaulay2 has various ways of dividing, but attempting to extending it to divide polynomialsleft me with a multitude of problems, highlighting how comlpli-

cated algebra can actually be.

5.2.1 Univariate Division Algorithm

The first algorithm I tried writing was A.1. This is basic univariate polynomail division, something that most high schoolers learn. The algorithm is rather simple. Take the leading term of the numerator and divide it by the denominator. Add the quotient to the result and subtrac the quotient times the denominator from the numerator. Repeat until the numerator is zero or the degree of the denominator exceeds that of the numerator. None of the ordering issues or really anything at all is difficult in this case. The built in division operator works without any issues. Filled with confidence, the multivariate algorithm was attempted.

5.2.2 Multivariate Division Algorithm

The idea behind the multivariate division algorithm is not much different that the univariate one. Divide a polynomial p by an array of polynomials gs. If the leadterm of p cannot be divided by the leadterm of the first polynomial of gs, move on to the next until one is that can is found. If no leading term of gs can divide the leading term of p, then the leading term is subratracted from p and the algorithm marches to the next polynomial. Seems easy enough, but there is quite a lot of complexity hiding between the lines. The first complexity had to do with the fact that there are more dimensions. Dividing with one dimension is easy enough, but when more are introduced, a way to organize the different dimesions is required. MonomialOrdering is introduced to solve this problem.

Another issue was how to divide the leadmonomials. With the addition of the higher dimension, the regular division was giving fractions of rings. This is an issue as the algorithm wants to remain within the ring itself, not outside of the ring. Foolishly, I decieded to try and create a leadingTermDivision function that decomposed the polynomials into lists of monomials, coefficients, and exponents. The idea was simple. Macaulay2 generates a list of the exponents taking the ring elements into account. So if the ring as $Q[x, y, z]$, the polynomial $x^2 * z$ would be decomposed into $\{2, 0, 1\}$. By creating these lists and subtracting the exponents from each other, the division has occured. All that would be left would be to recreate the polynomial using the generators of the ring and raise them to the exponent in the listed order. However, here my lacking understanding of Macaulay2 and more generally ring algebra can be seen. The generators of the ring, and the ring itself

are different. In line 63 of A.2, toList gens ring p is used. Here, we find the ring of polynomial p, find the generators of the ring, and then create a list of it. However, when later applying the exponential list to the generators, the returned polynomial is not an element of the ring that the function caller is using. It simply has the same variables.

Instead of recreating a way to divide monomials or using fractions of rings, there is a division that ensures the result remains in the ring. It does so by using division with remainder. With this realization, the algorithm was ready to be implemented. By first checking if the modulo of the leadmonomial would be 0, we can ensure that the ring division would not produce a remainder. So if the modulo did produce a remainder, the next gs was considered. If there was no remainder, essentially the univariate case is established.

One important note here, which justifies the use of Gröbner bases is that the order of the list gs matter significantly. Different ordering of the gs list generates different results. A test showing this can be foudn in A.2. Gröbner bases eliminates this reliance on the order which is desirable.

5.3 The Buchberger Algorithm

5.4 Solving Systems of Polynomials

A Macaulay2 source files

A.1 Algorithm121.m2

```
univariatePolynomialDivision = (p,g) -> (
    q := 0;
    r := p;
    while (r != 0_R and degree(g) < degree(r)) do (
        t := leadTerm(r) / leadTerm(g);
        q = q+t;
        r = r-t*g;
        );
    (q,r)
    )

runUnivariateTest = () -> (
    R = QQ[x];
    p := x^6 - 1;
    g := x^4 - 1;

    (resultQ, resultR) := univariatePolynomialDivision(p,g);

    print(resultQ == x^2);
    print(resultR == x^2 - 1);
)
```

Listing 1: Macaulay2 script Algorithm121.m2

A.2 algorithm123.m2

```
multivariatePolynomialDivision = (p, gs) -> (
    r := 0_(ring p);
    f := p;
    qs := for i from 0 to #gs-1 list 0_(ring p); --generating
        dummy variable for result

    while f != 0_(ring p) do (
        i := 0;
        divisionOccured := false;
        --print "Outer  loop";

        while (i < #gs and divisionOccured == false) do (
```

```
12              --print "Inner loop";
13              lmf := leadMonomial f;
14              lmg := leadMonomial (gs#i);
15
16              if (lmf % lmg == 0) then (
17                  monomialQuotient := lmf // lmg;
18
19                  coefficientQuotient := leadCoefficient f /
                        leadCoefficient gs#i;
20
21                  t := coefficientQuotient * monomialQuotient;
22
23                  qs = replace(i, qs#i + t, qs);
24                  f = f - t*gs#i;
25
26                  divisionOccured = true;
27                  --print "we were divisible";
28              )
29              else (
30                  i = i+1;
31
32                  --print "not divisible, function further along";
33              )
34          );
35
36          if divisionOccured == false then (
37              r = r + leadTerm(f);
38              f = f- leadTerm(f);
39              --print "division did not occur, subtracting leading
                    term and continuing";
40          );
41      );
42      return (qs, r)
43  )
44
45  -- This below code is useless, it was an attempt to get around the
        fact that i couldnt devide with the leadTerm.
46  -- However, I needed to seperate out the divisions as well as use
        "//" instead of "/"
47  leadingTermDevision = (p,g) -> (
48      lmP := leadMonomial(p);
49      lmG := leadMonomial(g);
```

```
50      lcP := leadCoefficient(p);
51      lcG := leadCoefficient(g);
52      exponentsP := flatten exponents lmP;
53      exponentsG := flatten exponents lmG;
54
55      divisible := all(#exponentsP, i -> exponentsP#i >=
            exponentsG#i); --checking that all exponents are greater or
            equal
56      result := 0_(ring p);
57
58      if divisible then(
59          coefficientResult := lcP / lcG;
60
61          exponentsQ := for i from 0 to (#exponentsP-1) list
                (exponentsP#i - exponentsG#i); --doing the division
62
63          ringVariables := toList gens ring p;
64
65          Q := product(#ringVariables, i -> ringVariables#i ^
                (exponentsQ#i)); --generating the the polynomial
66
67          result = coefficientResult * Q;
68      );
69
70      return (divisible, result);
71  )
72
73  runMultimultivariateTest = () -> (
74      R = QQ[x,y,MonomialOrder=>Lex];
75      p := -x^2*y-x^2+x*y+y^3;
76      g1 := x*y-x+y-1;
77      g2 := x+y+1;
78
79      (resultG1, resultRemainder1) :=
            multivariatePolynomialDivision(p,{g1,g2});
80      (resultG2, resultRemainder2) :=
            multivariatePolynomialDivision(p,{g2,g1});
81
82      print "-----------{g1,g2}------------";
83      print resultG1;
84      print resultRemainder1;
85      print (resultG1#0 == -x+4);
```

```
86      print (resultG1#1 == -2*x+5);
87      print (resultRemainder1 == y^3-9*y-1);
88
89      print "-----------{g2,g1}------------";
90      print resultG2;
91      print resultRemainder2;
92      print (resultG2#1 == 0);
93      print (resultG2#0 == -x*y-x+y^2+3*y+1);
94      print (resultRemainder2 == -4*y^2-4*y-1);
95  )
```

Listing 2: Macaulay2 script algorithm123.m2

A.3 algorithm124.m2

```
1   load "algorithm123.m2";
2
3   buchberger = ps -> (
4       g := ps;
5       gHat := {0_(ring  ps#0)};
6
7       while g != gHat do (
8           gHat = g;
9
10          -- generating all the possible pairs by looping through i
               and then from j -> end
11          polynomialPairs := flatten for i from 0 to #gHat-1 list
               for j from i+1 to #gHat-1 list (gHat#i, gHat#j);
12
13          i := 0;
14          while i < #polynomialPairs do (
15              p := (polynomialPairs#i)#0;
16              q := (polynomialPairs#i)#1;
17
18              xDelta := lcm(leadMonomial(p), leadMonomial(q));
19
20              s := xDelta // leadTerm(p) * p - xDelta // leadTerm(q)
                   * q;
21
22              (qs, r) = multivariatePolynomialDivision(s, gHat);
23
24              if r != 0_(ring ps#0) then (
```

```
                g = append(gHat,r);
                );

            i = i+1;
            );
        );
    print g;
    return g;
    )

testAlgorithm = () -> (
    R = QQ[x,y, MonomialOrder => GLex];

    p1 := x^2 + y;
    p2 := x^4 + 2*x^2*y + y^2 + 3;

    myBasis := gb ideal (buchberger({p1, p2}));
    correctBasis := gb ideal(p1, p2);
    areTheyEqual := myBasis == correctBasis;

    print "My basis:";
    print myBasis;

    print "Built-in basis:";
    print gens correctBasis;

    print "Are they equal?";
    print areTheyEqual;
    )

--again, this is not needed and M2 implements LCM.
findXDelta = (p,q) -> (
    alpha := flatten exponents leadMonomial(p);
    gamma := flatten exponents leadMonomial(q);

    delta := for i from 0 to (#alpha-1) list
        (max(alpha#i,gamma#i)); --getting the max of each variables
        exponent

    ringVariables := toList gens ring p;
    xDelta := product(#ringVariables, i -> ringVariables#i ^
        (delta#i)); -- reconstructing the variable
```

```
64
65      return xDelta
66      )
67
68  testGamma = () -> (
69      R := RR[x,y, MonomialOrder => GLex];
70      p := x^3*y^2-x^2*y^3+x;
71      q := 3*x^4*y +y^2;
72
73      xDelta := findXDelta(p,q);
74      xDeltaShouldBe := x^4*y^2;
75
76      print "-----------gamma results------------";
77      print xDelta;
78      print xDeltaShouldBe;
79      print("the result is " | toString(xDelta == xDeltaShouldBe))
80      )
```

Listing 3: Macaulay2 script algorithm124.m2

A.4 algorithm161.m2

```
1   solveSystemOfPolynomialEquations = (theRing, theIdeal) -> (
2       varsToLoopOver := gens theRing;
3
4      -- get the list of eigenvalues
5       eigenList := for var in varsToLoopOver list (
6           qMatrix := solveQMatrix(theRing, theIdeal, var);
7
8            --ensure that the eigenvector can be solved and save it
                 to the list
9            toList eigenvalues(sub(qMatrix,CC))
10          );
11
12      -- use fold to go through all elements in the eigenList
           outwards
13      candidatePoints := fold((L1, L2) -> (
14              --use table to create a table of all the pairs and
                   flatten down
15              flatten table(L1, L2, (a, b) -> (
16                      --if we are past first instance, we append
                           instead of create a list
```

```
                    if instance(a, List) then a | {b} else {a, b}
                    )
                )
            ),
        eigenList);

    epsilon := 1e-6;
    functionList := flatten entries gens theIdeal;

    validSolutions := select(candidatePoints, pt -> (
            -- ensure that x_n maps to the the nth point
            subMap = apply(#varsToLoopOver, i -> varsToLoopOver#i
               => pt#i);
            -- sub in the values
            all(functionList, p -> abs(sub(p, subMap)) < epsilon)
            ));

    -- something is wrong with my solveQMatrix which results in
       cluttered returns.
    -- so for now just to check if it works, we remove the
       duplicates
    uniqueSolutions := {};
    scan(validSolutions, v -> (
            if not any(uniqueSolutions, u -> (
                    dist := sqrt(sum(0..#v-1, i -> abs(v#i -
                       u#i)^2));
                    dist < epsilon
                    )) then uniqueSolutions =
                       append(uniqueSolutions, v);
            ));

    uniqueSolutions
    )


testAlgorithm2D = () -> (
    R := CC[x, y];
    I := ideal(x^2 + y^2 - 5, x*y - 2);

    myResults := solveSystemOfPolynomialEquations(R, I);
    solution := {{-2, -1}, {-1, -2}, {2, 1}, {1, 2}};

```

```
    print ("The 2D test output: " | toString  myResults);

    print ("Verified solution: " | toString solution);
)

testAlgorithm4D = () -> (
    R4 := CC[x1, x2, x3, x4];

    p1 := x1^2 + x2^2 - 2;
    p2 := x1 - x2;
    p3 := x3^2 + x4^2 - 13;
    p4 := x3*x4 - 6;
    I4 := ideal(p1, p2, p3, p4);

    results4D := solveSystemOfPolynomialEquations(R4, I4);

    if #results4D == 8 then (
        print "SUCCESS: Found all 8 points in 4D space.";
    ) else (
        print ("FAILURE: Expected 8 points, but found " |
            #results4D);
    );
)

solveQMatrix = (theRing, theIdeal, thePolynomial) ->(
    q := theRing / theIdeal;

    basisList := flatten entries basis q;
    polyBasis := for b in basisList list lift(b,theRing); --basis
        to R

    rows := for b in polyBasis list (
        currentPoly := thePolynomial * b;
        r := currentPoly % theIdeal;

        coeffsMatrix := last coefficients(r, Monomials =>
            polyBasis);
        flatten entries coeffsMatrix
        );

    return transpose matrix rows;
    )
```

```
93
94 print "-------2D CASE-------";
95 testAlgorithm2D();
96 print "------4D CASE---------";
97 testAlgorithm4D();
98 print "-------DONE-------";
```

Listing 4: Macaulay2 script algorithm161.m2

A.5 BookExample2.1.1.m2

```
1  R = QQ[x_1..x_4, MonomialOrder => Lex]
2
3  p1 = x_1-x_2**2+x_3
4
5  p2 = x_1**2+x_2*x_3**2+x_4
6
7  I = ideal(p1,p2)
8
9  G = gens gb I
10
11 polyToDivide = x_1**2+x_2**2+x_3**2+x_4**2-1
12
13 rem = polyToDivide%I
14
15 print("the rem is: ")
16 print(rem)
```

Listing 5: Macaulay2 script BookExample2.1.1.m2

A.6 BookExample2.2.m2

```
1  R = QQ[x_1,x_2,x_3]
2
3  p = (1/2)+x_1^2*x_2*x_3^4+(6/5)*x_1*x_3+2
4
5  -- output is a polynomial in R
```

Listing 6: Macaulay2 script BookExample2.2.m2

A.7 BookExample222.m2

```
R = frac(QQ[a]/ideal(a^2-2))[x_1,x_2,x_3]

substitute(a^2, R)
-- output is that a^2 = 2

p = x_1 + x_2 + a*x_3
```

Listing 7: Macaulay2 script BookExample222.m2

A.8 BookExample231.m2

```
R = QQ[z]

p = z^4+3*z^3-3*z^2+3*z-4

pc = -2*z^4+3*z^3-6*z^2+5*z-4

g = z^2+1

(q,r)= quotientRemainder(p,g)

(qc, rc) = quotientRemainder(pc,g)
```

Listing 8: Macaulay2 script BookExample231.m2

A.9 BookExample232.m2

```
R=QQ[c_1,c_2,c_3,c_4, MonomialOrder=>Lex]
M = matrix{{1,-2,0,-1},{4,0,7,3}, {7,-6,7,0}}
C = matrix{{c_1},{c_2},{c_3},{c_4}}
I = ideal(M*C)
GI = value toString gens gb I
(Ce,Me) = value toString coefficients(GI,
    Monomials=>{c_1,c_2,c_3,c_4})

Mr = value toString transpose Me

--powerful way to solve the row reduced form
inverse(substitute(submatrix(Mr, {0,1}),QQ))*Mr
```

Listing 9: Macaulay2 script BookExample232.m2

A.10 documentationConditionals.m2

```
isEven = i -> i % 2 == 0

oddOrEven = i -> (
    if isEven i then "even"
    else "odd")
```

Listing 10: Macaulay2 script documentationConditionals.m2

A.11 Example1612.m2

```
runExample = () -> (
    R = RR[z];

    g := z^3-3*z^2+2*z;

    I = ideal g;

    q := R / I;

    basisList := flatten entries basis q;
    polyBasis := for b in basisList list lift(b, R); -- ensuring
        that the actualy polynomials exists in R and not only in q

    rows := for b in  basisList list (
        currentPoly = z *lift(b,R);

        r := currentPoly % I;

        --taking #1 here since coefficients returns a bundle of
            want the coefficients, no the monomials
        coeffsMatrix := (coefficients(r, Monomials =>
            polyBasis))#1;
        flatten entries coeffsMatrix
        );

```

```
    zmatrix := matrix rows;

    finalZMatrix := zmatrix^2+zmatrix+id_(target zmatrix);
    return finalZMatrix;
)


 runExampleWithFunction = () -> (
    R = RR[z];

    g := z^3-3*z^2+2*z;

    I = ideal g;

    load "algorithm161.m2";

    zmatrix := solveQMatrix(R, I, z);

    finalZMatrix := zmatrix^2+zmatrix+id_(target zmatrix);
    return finalZMatrix;
)

testFunction = () -> (
    preBuiltMatrix := runExample();
    matrixFromFunction := runExampleWithFunction();

    print "Matrix from the example";
    print preBuiltMatrix;

    print "Matrix using refactored function";
    print matrixFromFunction;
 )
```

Listing 11: Macaulay2 script Example1612.m2

A.12 Example1613.m2

```
runExample = () -> (
    load "algorithm161.m2";

    R = RR[x,y];
    f := (y^2+x-3, x^2-4*x*y-y^2-10*y+15);
```

```
6       I := ideal f;
7
8       regMatrix := solveQMatrix(R,I,1);
9       xMatrix := solveQMatrix(R,I,x);
10      xyMatrix := solveQMatrix(R,I,x*y);
11      yMatrix := solveQMatrix(R,I,y);
12
13      --only care about the x,y individual ones
14      xEigens := (eigenvectors sub(xMatrix,CC))#0;
15      yEigens := (eigenvectors sub(yMatrix,CC))#0;
16
17      varietyList := for i from 0 to  #xEigens-1 list {xEigens#i,
            yEigens#i};
18      print varietyList;
19      )
```

Listing 12: Macaulay2 script Example1613.m2

References

[DAC10] Donal B O'Shea David A Cox, John Little. *Ideals, Varieties, and Algorithms: An Introduction to Computational Algebraic Geometry and Commutative Algebra*. Springer Publishing Company, third edition, November 2010.

[DF03] David S. Dummit and Richard M. Foote. *Abstract Algebra*. Wiley, Hoboken, NJ, USA, 3 edition, 2003.

[Diand] Jaime Dianzon. Keeping it real. GeoGebra interactive construction, n.d. Available at: https://www.geogebra.org/m/n8hauvp6.

[Gal25] Jean H. Gallier. Basics of affine geometry. In Jean H. Gallier, editor, *Geometric Methods and Applications for Computer Science and Engineering*, pages 7–63. Springer, 2025. Chapter 2 from online course materials. URL: https://www.cis.upenn.edu/~cis6100/geombchap2.pdf.

[Lat] Lattice and boolean algebra 2.1 algebra 2.2 lattice. URL: https://api.semanticscholar.org/CorpusID:18433539.

[LM21] Antonio Tornambè Laura Menini, Corrado Possieri. *Algebraic Geometry for Robotics and Control Theory*. WORLD SCIENTIFIC (EUROPE), 2021. arXiv:https://www.worldscientific.com/doi/pdf/10.1142/q0308.

[Wei70] Louis M. Weiner. *Introduction to Modern Algebra*. Harcourt, Brace & World, New York, NY, USA, 1970.